Shorten Path

AlgoExpert · Hard · Stacks

The problem

- Simplify a Unix path (absolute or relative), resolving "." and ".." and duplicate slashes.
- Leading ".." are kept for relative paths and dropped at the root of absolute paths.

Intuition

- Process path tokens with a stack: dots are ignored, double dots pop, names push.

Brute force vs optimal

- String replacement loops → fragile, $O(n^2)$
- Stack of path tokens → $O(n)$
- Handle absolute vs relative paths

Approach

- 1. Split on slashes
- 2. Skip empty tokens and single dots
- 3. Double dot pops, or is kept for relative paths
- 4. Join back

Worked example

stack (top ↑)



token 'foo': push

'foo': push

Worked example

stack (top ↑)



token 'test': push

'test': push

Worked example

stack (top ↑)



token 'test': push

'test': push

Worked example

stack (top ↑)



token '..': pop

'..': pop

Worked example

stack (top ↑)



token 'bar': push

'bar': push

Worked example

```
/foo/bar/baz
```

Join the stack

Java solution

```java
public class ShortenPath {
    public static String shortenPath(String path) {
        boolean abs = path.startsWith("/");
        ArrayDeque<String> st = new ArrayDeque<>();
        for (String t : path.split("/")) {
            if (t.isEmpty() || t.equals(".")) continue;
            if (t.equals("..")) {
                if (!st.isEmpty() && !st.peekLast().equals("..")) st.pollLast();
                else if (!abs) st.addLast("..");
            } else st.addLast(t);
        }
        String joined = String.join("/", st);
        return abs ? "/" + joined : joined;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- '..' at root of absolute path → ignore
- Relative paths keep leading '..'
- Empty tokens from '//'

Interview tips & follow-ups

- Same as Simplify Path

Recap

- Process path tokens with a stack: dots are ignored, double dots pop, names push.
- Complexity: O(n)